Software Requirements Specification (SRS)

Project: Online Quiz System — console-based multiple-choice quiz game with scoring (C / C++)

Version: 1.0

Authors: << Student name / SRN >>

Date: 06-09-2026

Status: Draft for review

Revision history

Version	Date	Author	Change summary	Approval
1.0	06-09-2026	Madhusudhan	1. Introduction 2. Overall description	
2.0	06-09-2026	Sarah	3. External interface requirements 4. System features (detailed)	
3.0	06-09-2026	Thanvitha	5. Non-functional requirements 6. Quality attributes & Acceptance tests	
4.0	06-09-2026	Jishnu	7. System models and diagrams 8. Requirements Traceability Matrix (RTM)	

Approvals

Role	Name	Signature / Email	Date
Course Coordinator			
Student (Author)			

Table of Contents

1. Introduction
2. Overall description
3. External interface requirements
4. System features (detailed)
5. Non-functional requirements (detailed)
6. Quality attributes & Acceptance tests
7. System models and diagrams (UML use-case)
8. Requirements Traceability Matrix (RTM)

1. Introduction

1.1 Purpose

This document is a Software Requirements Specification (SRS) for the Online Quiz System, a single-user, console-based multiple-choice quiz application implemented in C / C++. It defines the functional and non-functional requirements, the external interfaces, the security objectives and requirements, and the verification criteria against which the delivered program will be assessed. The intended readers are the developer, the reviewing instructor, and any evaluator running the acceptance tests.

1.2 Scope

The system presents multiple-choice questions to a player through a text terminal, accepts a keyed answer per question, scores the attempt, and persists results to a local file so that a leaderboard can be shown. An authenticated administrator mode allows the question bank to be listed, added to, edited, and deleted. All state is held in plain local files; there is no database, no network component, and no concurrent multi-user access.

Explicitly out of scope: graphical or web user interfaces, networked or client–server play, user account registration with per-user credentials beyond the single administrator passphrase, question types other than single-correct-answer multiple choice, and any form of adaptive or randomised question generation beyond shuffling an existing bank.

1.3 Audience

Developer, code reviewer, QA / test author, course instructor, and assessment evaluator

1.4 Definitions and Acronyms

Term	Meaning
MCQ	Multiple-Choice Question
CLI	Command-Line Interface
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
RTM	Requirements Traceability Matrix
Question Bank	The local collection of quiz questions, answer options, and correct answers
Attempt	One complete run of the quiz by a player

2. Overall description

2.1 Product perspective

The Online Quiz System is a self-contained console executable. It is not a component of a larger system and exposes no API. It reads two data files at start-up — the question bank and the results file — and writes the results file and an administrator audit log. The program is single-process and single-threaded; the only concurrency consideration is that two copies of the program started against the

same data directory must not corrupt each other's writes, which is handled by atomic replacement rather than by locking.

2.2 Major product functions

- Player identification and session start
- Category and question-count selection
- Randomised presentation of MCQs with option shuffling
- Bounded, validated answer input with an optional per-question time limit
- Scoring (flat or difficulty-weighted) and end-of-session score report
- Per-question answer review after submission
- Result persistence and top-N leaderboard display
- Administrator-authenticated question-bank management (list, add, edit, delete)
- Question-bank integrity verification and administrator audit logging

2.3 User roles and characteristics

- Player — any user with basic terminal familiarity. Expects unambiguous prompts, tolerance of mistyped input, and a clear final score. No prior knowledge of the file formats is assumed.
- Administrator — the instructor or question author. Knows the question format, needs to edit the bank quickly, and is trusted with the administrator passphrase.
- Evaluator — runs the acceptance test suite and inspects data files directly; needs the on-disk formats to be human-readable and documented.

2.4 Operating environment

- Any POSIX-compliant system (Linux, macOS) or Windows 10+ with a terminal of at least 80 columns × 24 rows.
- Toolchain: GCC 9+ / Clang 12+ / MSVC 2019+; C11 or C++17.
- Storage: local filesystem, read-write access to the program's data directory. Total data footprint under 1 MB for a 1000-question bank.
- No network connectivity is required or used at any point.

2.5 Constraints and assumptions

- Standard library only, plus platform headers where a portable equivalent does not exist (for example, non-blocking input for the question timer). Third-party libraries are not permitted.
- Data files are plain text so that they can be inspected and diffed by the evaluator.
- Exactly one correct option per question; partial credit is not supported.
- The per-question timer requires non-blocking console input, which has no ISO C equivalent; the implementation is expected to isolate this behind a small platform-abstraction module (poll/select on POSIX, kbhit on Windows).
- The administrator passphrase protects against casual tampering by another user of the same machine. It is not a defence against a user who can already write to the data files directly; that limitation is stated explicitly in section 5.1.1.

3. External interface requirements

3.1 User interfaces

A single text-mode interface. The main menu offers numbered choices (Take Quiz, View Leaderboard, Administrator Mode, Help, Exit) selected by typing a digit and pressing Enter. Question screens display the stem, four labelled options (A–D), the question index ("Question 4 of 10"), and the remaining time when the timer is enabled. Every prompt states the accepted input set. Invalid input produces a one-line diagnostic and re-prompts without consuming an attempt. All output fits within 80 columns; long question stems are word-wrapped, not truncated.

3.2 Hardware Interfaces

- Keyboard for text/numeric input
- Monitor or terminal display for textual output
- Standard local storage when a file-based question bank is used

3.3 Software Interfaces

- Operating system console/terminal
- Standard C/C++ runtime libraries
- Optional local text/CSV-style question file if the implementation does not embed questions directly in source code

3.4 Communications Interfaces

No network or inter-process communication interface is required for the base version of the Online Quiz System.

4. System features

Each requirement below carries acceptance criteria and a reference test case. Functional requirement IDs follow the pattern QUIZ-F.

4.1 Session start and identification

Description: establish who is playing and which privilege level the session runs at.

Req ID	Requirement (shall…)	Type	Prio.	Source / Stakeholder	Acceptance criteria / Test ref	Dependencies
QUIZ-F-001	The system shall prompt for a player name of 1–30 characters and accept only letters, digits, spaces, underscores and hyphens.	Functional	High	Evaluator	AC-001: A 30-character valid name is accepted; a 31-character or empty name is rejected with a re-prompt. Test: TC-SES-01	Feeds the results record
QUIZ-F-002	The system shall require an administrator passphrase before entering administrator mode and shall verify it against a stored salted hash.	Functional	High	Security	AC-002: Correct passphrase grants admin menu; incorrect passphrase is refused and the menu is not shown. Test: TC-SES-02	Depends on QUIZ-SR-002
QUIZ-F-003	The system shall disable administrator mode for the remainder of the process after 3 consecutive failed passphrase attempts and shall record the event in the audit log.	Functional	High	Security	AC-003: After 3 failures the admin option is rejected without prompting; one lockout line appears in admin.log. Test: TC-SES-03	Per-process only; a restart resets it

4.2 Question Presentation and Answer Input

Description: Present MCQs and safely collect one answer per question.

Req ID	Requirement (shall…)	Type	Priority	Acceptance Criteria / Test Case	Dependencies / Comments
OQS-F-006	The system shall present one question at a time with a visible question number.	Functional	High	AC: Only the active question is requested for input. Test: TC-QUIZ-01	Question bank loaded

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Case	Dependencies / Comments
OQS-F-007	The system shall display all answer options associated with the current question using consistent option labels.	Functional	High	AC: Every configured option for the current question is printed. Test: TC-QUIZ-02	Console output
OQS-F-008	The system shall accept an answer using the documented option format, such as A-D or 1-4.	Functional	High	AC: Valid option input is accepted and recorded. Test: TC-ANS-01	Keyboard input
OQS-F-009	The system shall reject input that does not correspond to a displayed option and shall prompt the player again without advancing the question.	Functional	High	AC: Invalid input does not change score or question index. Test: TC-ANS-02	Robust input handling
OQS-F-010	The system shall record exactly one accepted answer for each question in the current attempt.	Functional	High	AC: Each completed question has one stored response. Test: TC-ANS-03	Attempt state

4.3 Answer Evaluation and Scoring

Description: Evaluate responses and maintain a correct score.

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Case	Dependencies / Comments
OQS-F-011	The system shall compare each accepted answer with the stored correct answer for that question.	Functional	High	AC: Evaluation result matches the configured answer key. Test: TC-SCORE-01	Answer key
OQS-F-012	The system shall increment the score by one point for each correct answer and shall not increment it for an incorrect answer.	Functional	High	AC: Score changes only after a correct response. Test: TC-SCORE-02	OQS-F-011
OQS-F-013	The system shall display whether the submitted answer was correct or incorrect after evaluation.	Functional	Medium	AC: Feedback appears after every accepted answer. Test: TC-SCORE-03	OQS-F-011
OQS-F-014	The system shall show quiz progress using the current question number and total question count.	Functional	Medium	AC: Progress values are accurate for every displayed question. Test: TC-QUIZ-03	Question count
OQS-F-015	The system shall continue to the next question after the current answer has been evaluated, until no questions remain.	Functional	High	AC: All questions are asked exactly once in the base sequential mode. Test: TC-QUIZ-04	Session loop

4.4 Results and Session Control

Description: Complete the attempt and let the player replay or exit.

Req ID	Requirement (shall...)	Type	Priority	Acceptance Criteria / Test Case	Dependencies / Comments
OQS-F-016	The system shall calculate the final score after the last question has been evaluated.	Functional	High	AC: Final score equals the number of correct answers. Test: TC-RES-01	OQS-F-012
OQS-F-017	The system shall calculate the final percentage as (score / total questions) x 100.	Functional	High	AC: Percentage is mathematically correct; division by zero is prevented. Test: TC-RES-02	Total questions > 0
OQS-F-018	The system shall display a final result summary containing the player name, score, total questions, and percentage.	Functional	High	AC: All required result fields are displayed. Test: TC-RES-03	OQS-F-002, OQS-F-016, OQS-F-017
OQS-F-019	The system shall offer the player an option to replay the quiz after the result summary is displayed.	Functional	Medium	AC: Choosing replay starts a fresh attempt. Test: TC-CTRL-01	Result completed
OQS-F-020	The system shall terminate cleanly when the player chooses to exit.	Functional	High	AC: Program exits without crash or unfinished prompt. Test: TC-CTRL-02	Console process

4.5 Persistence and leaderboard

Description: keep results across runs and rank them.

Req ID	Requirement (shall…)	Type	Prio.	Source / Stakeholder	Acceptance criteria / Test ref	Dependencies
QUIZ-F-040	The system shall append one result record per completed or aborted session containing the ISO 8601 timestamp, player name, category, questions attempted, correct count, score and maximum score.	Functional	High	Evaluator	AC-040: results.csv grows by exactly one well-formed line per session. Test: TC-PER-01	Append mode; no rewrite of history
QUIZ-F-041	The system shall display a leaderboard of the top 10 results sorted by percentage descending, then by timestamp ascending for ties.	Functional	Medium	Player	AC-041: Two equal percentages appear with the earlier timestamp first. Test: TC-PER-02	Reads results.csv only
QUIZ-F-042	The system shall treat a missing results file as an empty result set, create it on first write, and continue after reporting an unreadable file rather than terminating.	Functional	Medium	Developer	AC-042: Deleting results.csv then running a session recreates it; a chmod 000 file produces a warning and a still-usable program. Test: TC-PER-03	Graceful-degradation case

5. Non-functional requirements (detailed)

NFRs are measurable quality constraints and are tied to verification activities

Req ID	Requirement	Category	Priority	Acceptance Criteria / Measurement
OQS-NF-001	For a local question bank of up to 1,000 questions, the application shall launch and present the first question within 2 seconds on a typical lab computer.	Performance	High	Measure elapsed startup time in 10 runs; each run must be <= 2 s. Test: TC-NF-PERF-01
OQS-NF-002	After a valid answer is entered, the application shall display evaluation feedback and the next prompt within 1 second under normal local execution.	Performance	Medium	Observed response time <= 1 s for 20 consecutive answers. Test: TC-NF-PERF-02
OQS-NF-003	The application shall not crash, terminate unexpectedly, or corrupt the score when given invalid menu/answer input during a 100-input robustness test.	Reliability	High	100 invalid/valid mixed inputs complete without abnormal termination. Test: TC-NF-REL-01
OQS-NF-004	All interactive prompts shall use consistent wording and clearly state the expected input format.	Usability	Medium	Usability checklist confirms start, answer, replay, and exit prompts are unambiguous. Test: TC-NF-UX-01
OQS-NF-005	The source code shall compile successfully using at least one standards-compliant C11 compiler or C++17 compiler on the target lab environment without source modification beyond build configuration.	Portability / Maintainability	Medium	Clean build succeeds with documented compiler command. Test: TC-NF-PORT-01

5.1 Security

5.1.1 Security objectives

This system holds no personal data and runs entirely on the local machine, so the realistic threat is not a remote attacker but a local user — typically a player — who wants to see the answers, alter their score, or crash the program with hostile input. The objectives below are scoped to that threat model.

- SO-1 (Integrity of assessment data): the question bank, the results file and the audit log shall be modifiable through the program only by an authenticated administrator, and unauthorised modification made outside the program shall be detectable.

- SO-2 (Memory safety against untrusted input): all data reaching the program — keyboard input and file contents alike — shall be treated as untrusted, so that no input can cause a buffer overflow, an out-of-bounds access or other undefined behaviour.
- SO-3 (Confidentiality of the administrator credential): the administrator passphrase shall not be recoverable from any file the program writes, from its output, or from its source.

Stated limitation: a local user with write access to the data directory can replace both the bank and its digest file, and a user who can read process memory can recover the passphrase while it is being verified. Defeating either would require OS-level access control or a trusted store, both outside the scope of a console coursework application. SO-1 therefore targets detection, not prevention.

5.1.2 Security requirements

Req ID	Requirement (shall…)	Type	Priority	Acceptance criteria / Test case ref
QUIZ-SR-001	All console and file input shall be read with length-bounded calls (fgets or std::getline into a std::string); gets, scanf("%s") and strcpy into fixed buffers shall not appear in the source.	Security	High	Source grep finds no banned call; a 1 MB input line at every prompt is truncated or rejected with no overflow under ASan. Test: TC-SEC-01
QUIZ-SR-002	The administrator passphrase shall be stored only as a salted hash with a per-installation random salt of at least 16 bytes, and shall never be echoed to the terminal or written to any log.	Security	High	No plaintext passphrase in any file or log; terminal echo disabled during entry. Test: TC-SEC-02
QUIZ-SR-003	Every field parsed from the question bank shall be validated against a documented maximum length (stem ≤ 512, option ≤ 256, category ≤ 64 characters) and records exceeding it shall be rejected, not truncated into a fixed buffer.	Security	High	A crafted bank with a 100 KB stem is rejected with a line-numbered diagnostic and no memory error. Test: TC-SEC-03
QUIZ-SR-004	No string originating from user input or from a data file shall be passed as the format argument of a printf-family function.	Security	High	Source review and -Wformat-security clean; a question stem containing %n and %s renders literally. Test: TC-SEC-04
QUIZ-SR-005	The system shall verify the SHA-256 digest of the question bank before use and shall refuse all write operations while the digest does not match.	Security	Medium	Tampered bank triggers read-only mode; admin add/edit/delete are unavailable. Test: TC-SEC-05
QUIZ-SR-006	Data files created by the system shall be given owner-only permissions (0600 on POSIX) and the program shall refuse to write through a symbolic link.	Security	Medium	Newly created files show mode 0600; replacing results.csv with a symlink causes a refusal, not a write through it. Test: TC-SEC-06

6. Quality attributes & acceptance tests

Exit criteria for acceptance:

- Every high-priority functional requirement is implemented and its acceptance criteria verified.
- No non-functional requirement fails: in particular, a clean sanitizer run (QUIZ-NF-002) and a warning-free build (QUIZ-NF-003) are blocking.
- All six security requirements pass; QUIZ-SR-001 to QUIZ-SR-004 are blocking, QUIZ-SR-005 and QUIZ-SR-006 may be waived with a written justification.
- The RTM in section 8 shows every requirement mapped to at least one executed test case with status A (accepted).

Acceptance test suites: TC-SES (session and authentication), TC-BNK (bank management), TC-QZ (quiz flow), TC-SCR (scoring), TC-PER (persistence and leaderboard), TC-INT (integrity and audit), TC-PERF, TC-REL, TC-PORT, TC-UX, TC-MNT (non-functional), TC-SEC (security).

Verification methods used: automated tests driving stdin with recorded transcripts compared against expected stdout; sanitizer and Valgrind runs; static analysis for complexity; manual inspection for the source-level security requirements.

7. System models and diagrams

7.1 Use-case diagram — player quiz session

The player-facing boundary. The player starts a session, answers questions, may abort, and views the score report and leaderboard. Option-input validation is included in every answer; the timeout handler extends validation because it only occurs when the configured limit expires (QUIZ-F-024). The question bank and results files act as a supporting actor at the read/append points.

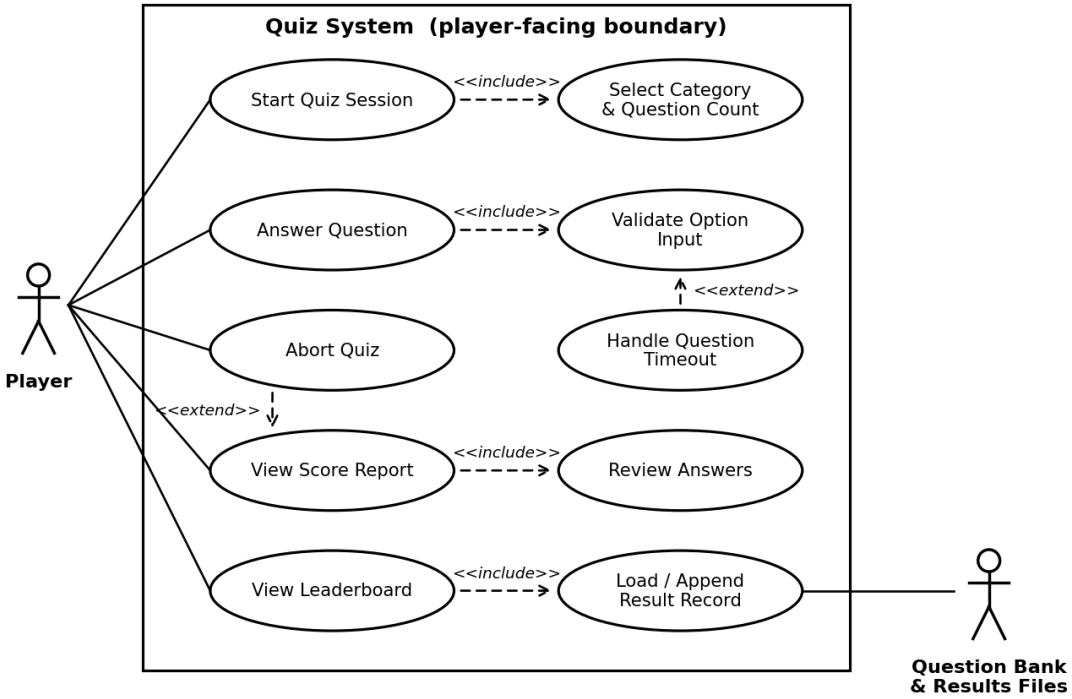


Figure 7.1 — Use-case diagram: player quiz session

7.2 Use-case diagram — question-bank administration

The administration boundary. Authentication (QUIZ-F-002) is a precondition of every bank-modifying use case; the lockout after three failures extends it (QUIZ-F-003). Add, edit and delete each include the atomic persist step, which in turn includes writing an audit record. Checksum verification runs against the same local file store.

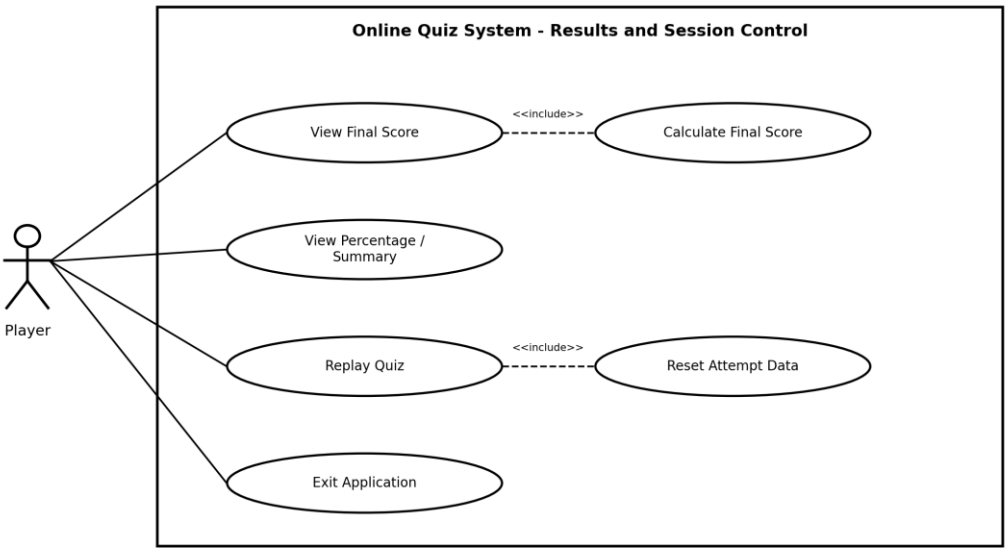


Figure 7.2 — Use-case diagram: question-bank administration

8. Requirements Traceability Matrix (RTM)

Req ID	Requirement Short Name	Section Ref	Module	Test Case(s)	Status (N/P/A)	Comments
OQS-F-001	Welcome / start	4.1	Main / UI	TC-INIT-01	N	
OQS-F-002	Capture player name	4.1	Input Module	TC-INIT-02	N	
OQS-F-003	Initialize attempt	4.1	Session Module	TC-INIT-03	N	
OQS-F-004	Load question bank	4.1	Question Bank	TC-QB-01	N	
OQS-F-005	Validate question records	4.1	Question Bank	TC-QB-02	N	
OQS-F-006	Present one question	4.2	Quiz Engine	TC-QUIZ-01	N	
OQS-F-007	Display options	4.2	Quiz Engine / UI	TC-QUIZ-02	N	
OQS-F-008	Accept answer	4.2	Input Module	TC-ANS-01	N	
OQS-F-009	Reject invalid answer	4.2	Input Validation	TC-ANS-02	N	
OQS-F-010	Record response	4.2	Session Module	TC-ANS-03	N	
OQS-F-011	Compare answer key	4.3	Scoring Module	TC-SCORE-01	N	
OQS-F-012	Update score	4.3	Scoring Module	TC-SCORE-02	N	
OQS-F-013	Answer feedback	4.3	UI / Scoring	TC-SCORE-03	N	
OQS-F-014	Show progress	4.3	Quiz Engine / UI	TC-QUIZ-03	N	
OQS-F-015	Advance questions	4.3	Quiz Engine	TC-QUIZ-04	N	
OQS-F-016	Final score	4.4	Results Module	TC-RES-01	N	
OQS-F-017	Percentage	4.4	Results Module	TC-RES-02	N	
OQS-F-018	Result summary	4.4	Results / UI	TC-RES-03	N	
OQS-F-019	Replay	4.4	Session Control	TC-CTRL-01	N	
OQS-F-020	Exit	4.4	Session Control	TC-CTRL-02	N	
OQS-NF-001	Performance target	5	Application / Build	TC-NF-PERF-01	N	
OQS-NF-002	Performance target	5	Application / Build	TC-NF-PERF-02	N	
OQS-NF-003	Reliability target	5	Application / Build	TC-NF-REL-01	N	
OQS-NF-004	Usability target	5	Application / Build	TC-NF-UX-01	N	
OQS-NF-005	Portability / Maintainability target	5	Application / Build	TC-NF-PORT-01	N	
OQS-SR-001	Security control	5.1	Input / Quiz / Scoring	TC-SEC-01	N	
OQS-SR-002	Security control	5.1	Input / Quiz / Scoring	TC-SEC-02	N	
OQS-SR-003	Security / Integrity control	5.1	Input / Quiz / Scoring	TC-SEC-03	N	
OQS-SR-004	Security control	5.1	Input / Quiz / Scoring	TC-SEC-04	N	
OQS-SR-005	Security / Integrity control	5.1	Input / Quiz / Scoring	TC-SEC-05	N	

Status legend: N = Not Run, P = Pass, A = Accepted / Approved.